

IBM University Engagement Project Submission

“Exploring the Power of Agentic AI with IBM Granite and LangFlow”

Project Tile – Scholar AI

Domain of Project – Education/Research

Student Name	Recent Passport Photo		Email ID	Mobile number [WhatsApp]
K S Sreeniranjana			sreeniranjana10ks@gmail.com	7592846122

Problem Statement –Research Agent

The Challenge

Students often spend significant time searching multiple websites for reliable academic information. Existing resources may provide incomplete, outdated, or scattered content, making it difficult to obtain quick and accurate learning support.

The Objective

Develop **Scholar AI**, an AI-powered educational assistant that provides instant, accurate, and personalized academic guidance through a simple conversational interface. The system aims to improve learning efficiency by delivering reliable responses in real time.

Proposed Solution

Proposed Solution

Scholar AI is an AI-powered educational assistant developed using **IBM Watsonx Orchestrate, IBM Granite 13B Chat v2, Python Flask, HTML, CSS, and JavaScript**. The application enables students to ask academic questions through a user-friendly web interface.

The frontend sends user queries to a Flask backend, which securely communicates with IBM Watsonx Orchestrate. Powered by IBM Granite, the AI processes each query and returns accurate, context-aware educational responses in real time. Scholar AI provides an interactive and efficient learning experience while reducing the effort required to search across multiple educational resources.

Technology Used

IBM Watsonx Orchestrate – Built and managed the Scholar AI agent workflow.

IBM Granite 13B Chat v2 – Generated intelligent, context-aware academic responses.

IBM IAM (OAuth) – Secured API access using bearer token authentication.

Python 3.11 & Flask – Developed the backend and integrated AI services.

HTML5, CSS3 & JavaScript – Created the responsive web-based user interface.

Requests & Flask-CORS – Enabled secure API communication between frontend and backend.

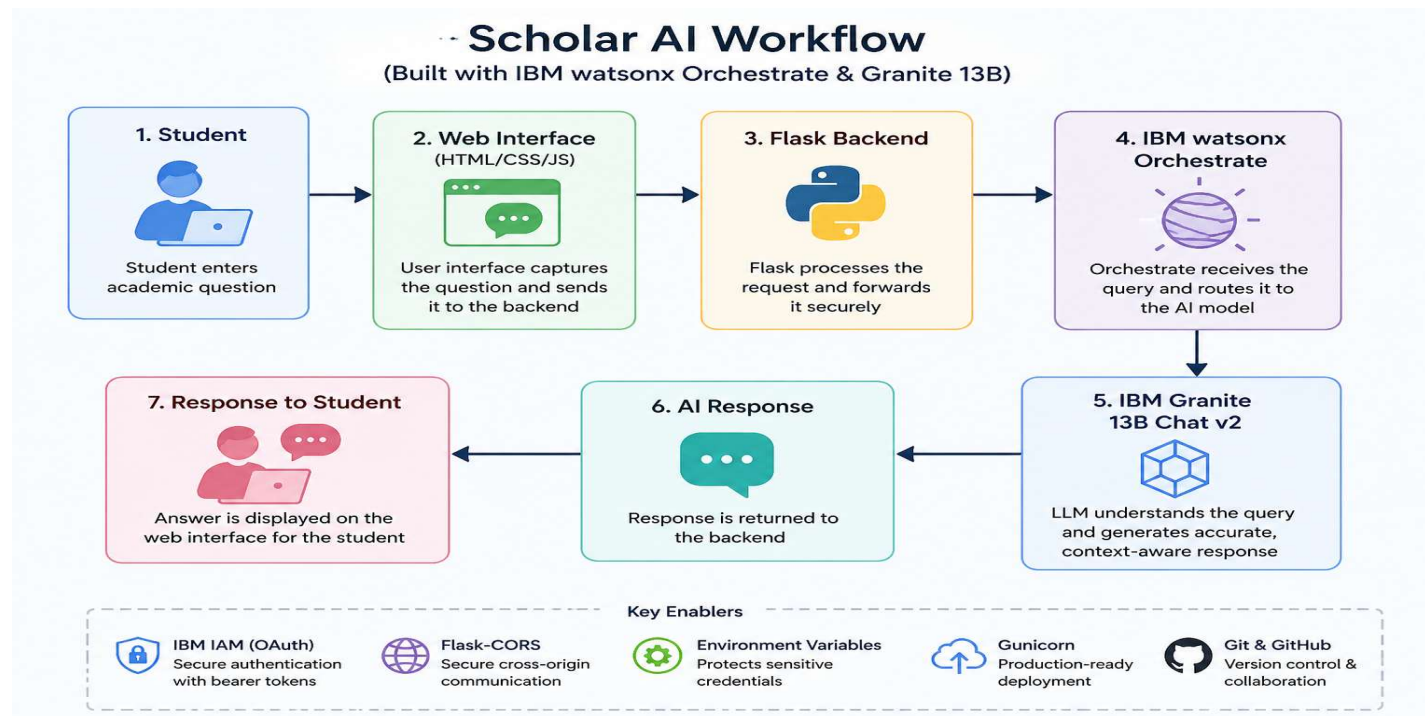
Git, GitHub & VS Code – Used for version control and application development.

Gunicorn & Environment Variables – Supported deployment and protected sensitive credentials.

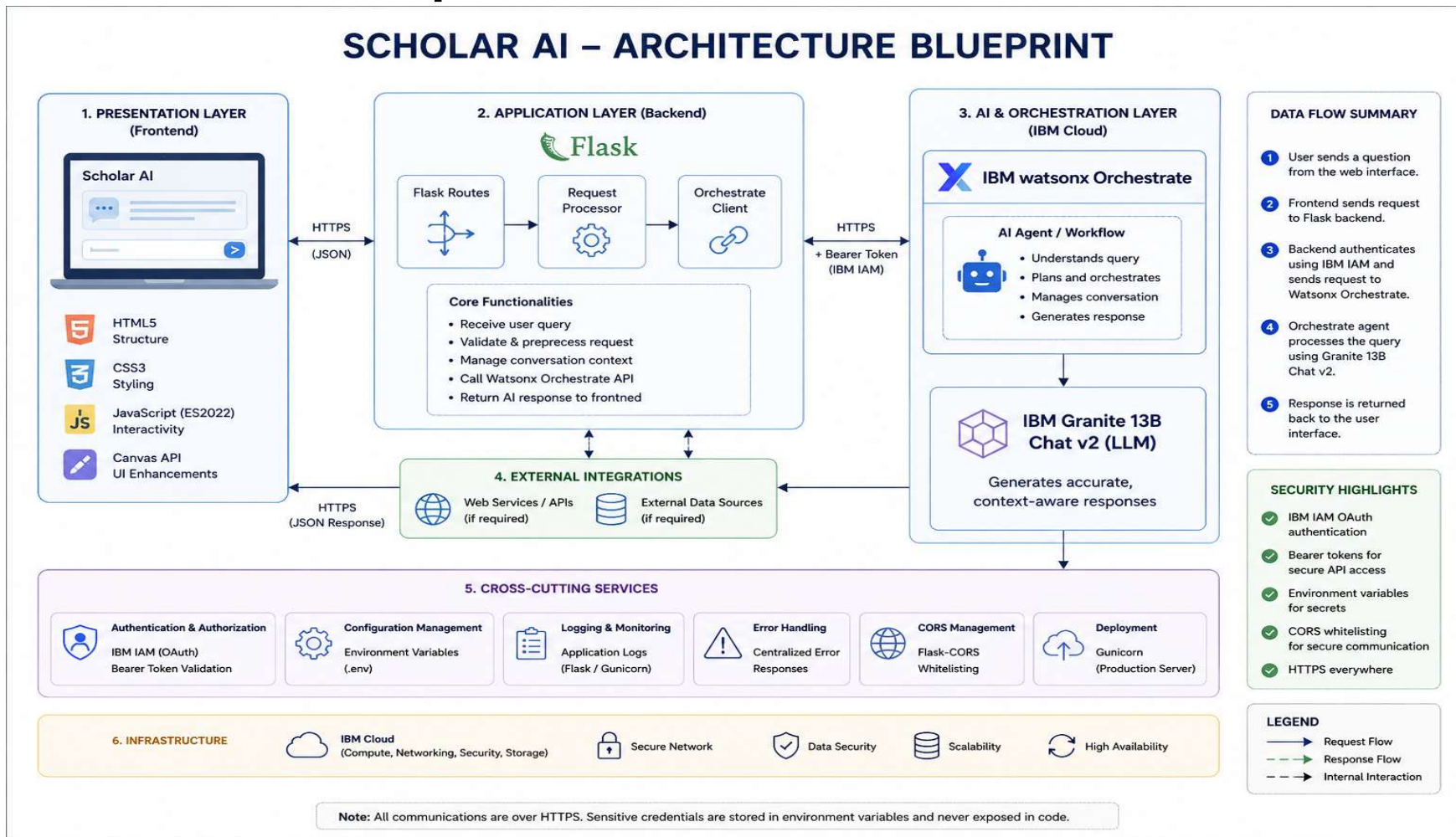
Watsonx Orchestrate Components Used

- **1. Chat Input** – Accepts academic questions entered by the user through the Scholar AI web interface.
- **2. AI Agent (Watsonx Orchestrate)** – Receives user queries, manages the workflow, and coordinates response generation.
- **3. IBM Granite 13B Chat v2** – Processes the query using natural language understanding and generates accurate academic responses.
- **4. API Integration** – Connects the Flask backend with IBM Watsonx Orchestrate using secure IBM IAM authentication.
- **5. Chat Output** – Displays AI-generated answers and explanations to the user in real time.

Workflow -

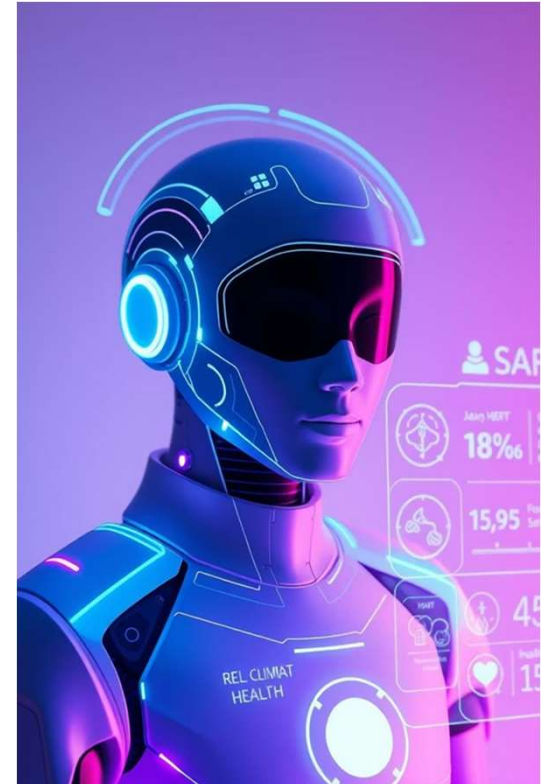


Architecture blueprint :



Role of Agentic AI in the solution

Agentic AI enables **Scholar AI** to function as an intelligent academic assistant by understanding user queries, reasoning over the context, and generating accurate responses. Using **IBM Watsonx Orchestrate** and the **IBM Granite 13B Chat v2** model, the system autonomously processes requests and delivers relevant educational assistance in real time. Instead of providing predefined answers, Agentic AI interprets natural language, manages the conversation flow, and produces context-aware responses based on the student's questions. This makes Scholar AI more interactive, efficient, and personalized. Overall, Agentic AI transforms Scholar AI into a smart learning companion that offers reliable, instant, and user-friendly academic support, enhancing students' learning experience.



Novelty and Uniqueness

Agentic AI Approach – Uses IBM Watsonx Orchestrate with IBM Granite to provide intelligent, context-aware academic assistance.

Real-Time Learning Support – Delivers instant and accurate responses through a conversational interface.

Secure Architecture – Implements IBM IAM authentication and secure API integration for reliable access.

User-Friendly Web Application – Built with Flask, HTML, CSS, and JavaScript for a simple and responsive learning experience.

Scalable Design – Can be extended with document analysis, multilingual support, and personalized learning features.

Git Hub Link

Github Link:- <https://github.com/sreeniranjana10ks-max/Scholar-AI.git>

.

.

.

Future Scope

Personalized Learning – Recommend study plans based on students' learning progress and preferences.

Multilingual Support – Enable interactions in multiple languages to improve accessibility.

Document Analysis – Allow users to upload PDFs and notes for AI-powered summarization and question answering.

Mobile Application – Develop Android and iOS apps for learning anytime, anywhere.

.

.

Certificates Earned:



In recognition of the commitment to achieve
professional excellence



K S SREENIRANJAN

Has successfully satisfied the requirements for:

Getting Started with Cybersecurity



Issued on: Jun 28, 2026
Issued by: IBM SkillsBuild

Verify: <https://www.credly.com/badges/4c17d671-b540-450b-af4b-37f22b737021>



Badges Earned :-

Getting Started with
Artificial Intelligence

IBM SkillsBuild



IBM

Getting Started with
Cybersecurity

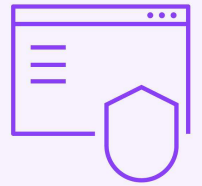
IBM SkillsBuild



IBM

Level Up Cybersecurity
with Generative AI

IBM SkillsBuild



Certificates Earned:

6/30/26, 8:21 PM

Completion Certificate | SkillsBuild

IBM **SkillsBuild**

Completion Certificate



This certificate is presented to

K S SREENIRANJAN

for the completion of

Lab: Troubleshoot Your Code Using IBM Bob

(ALM-COURSE_4071307)

According to the Adobe Learning Manager system of record

Completion date: 30 Jun 2026 (GMT)

Learning hours: 30 mins

IBM **SkillsBuild**

Completion Certificate



This certificate is presented to

K S SREENIRANJAN

for the completion of

Getting Started with Artificial Intelligence

(PLAN-E624C2604060)

According to the Your Learning Builder - Plans system of record

Completion date: 28 Jun 2026 (GMT)

IBM **SkillsBuild**

Completion Certificate



This certificate is presented to

K S SREENIRANJAN

for the completion of

Getting Started with Cybersecurity (Earn a credential!)

(PLAN-C7EE7CC95370)

According to the Your Learning Builder - Plans system of record

Completion date: 28 Jun 2026 (GMT)

Certificates Earned:

6/30/26, 8:19 PM

Completion Certificate | SkillsBuild

IBM **SkillsBuild**

Completion Certificate



This certificate is presented to

K S SREENIRANJAN

for the completion of

Level Up Cybersecurity with Generative AI

(ALM-COURSE_3947352)

According to the Adobe Learning Manager system of record

Completion date: 30 Jun 2026 (GMT)

Learning hours: 1 hr 30 mins

IBM **SkillsBuild**

Completion Certificate



This certificate is presented to

K S SREENIRANJAN

for the completion of

Exploring Quantum Computing

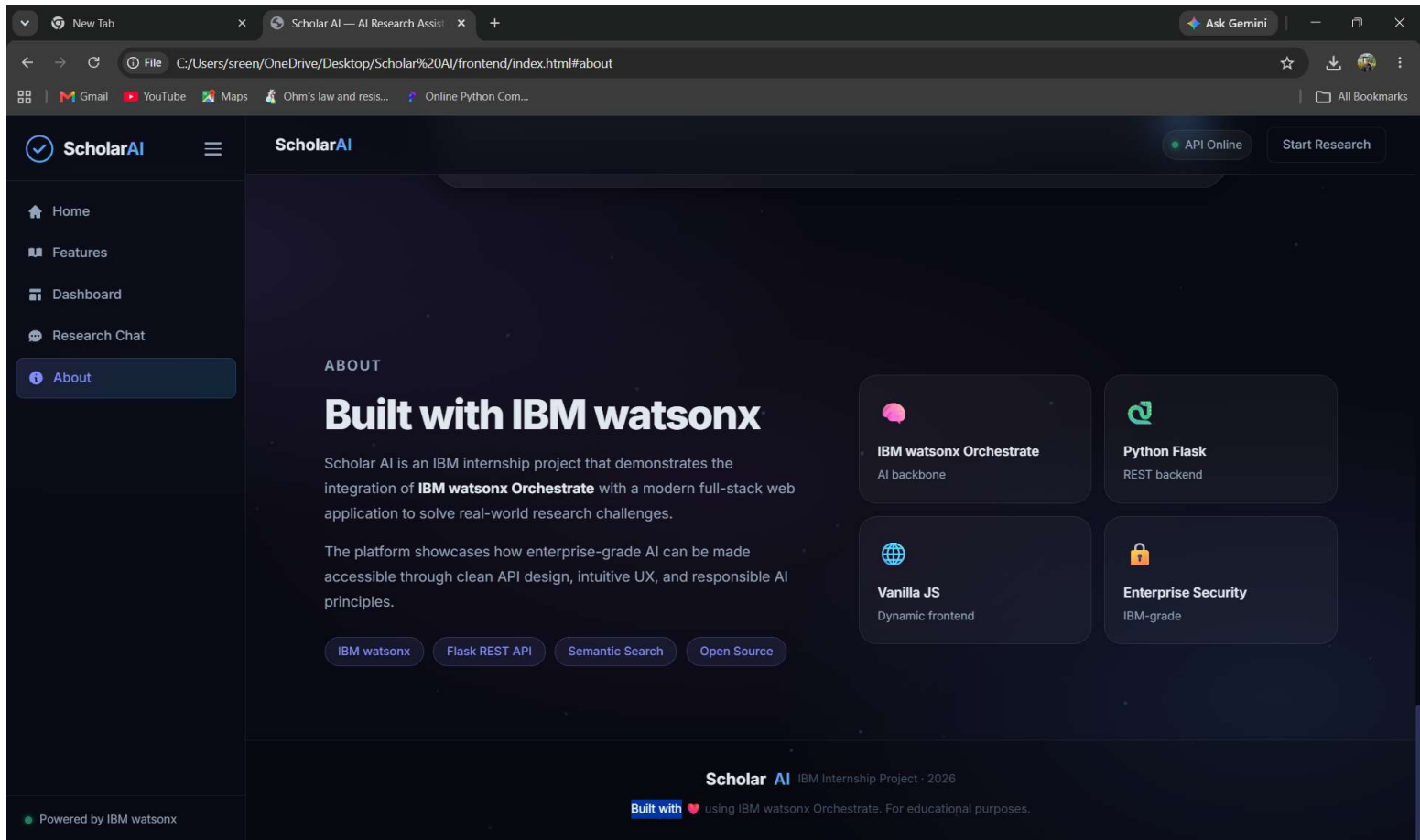
(ALM-COURSE_3825253)

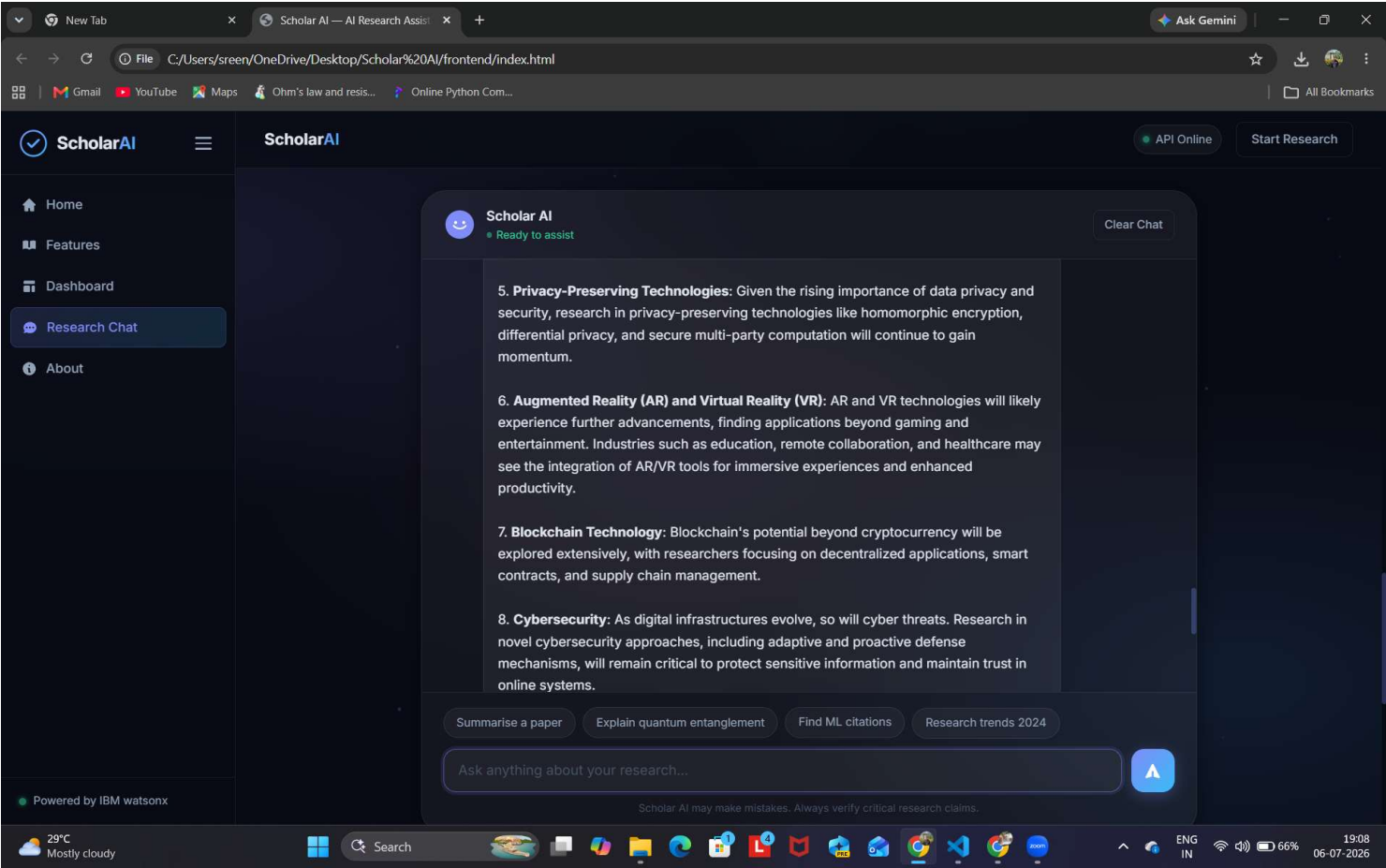
According to the Adobe Learning Manager system of record

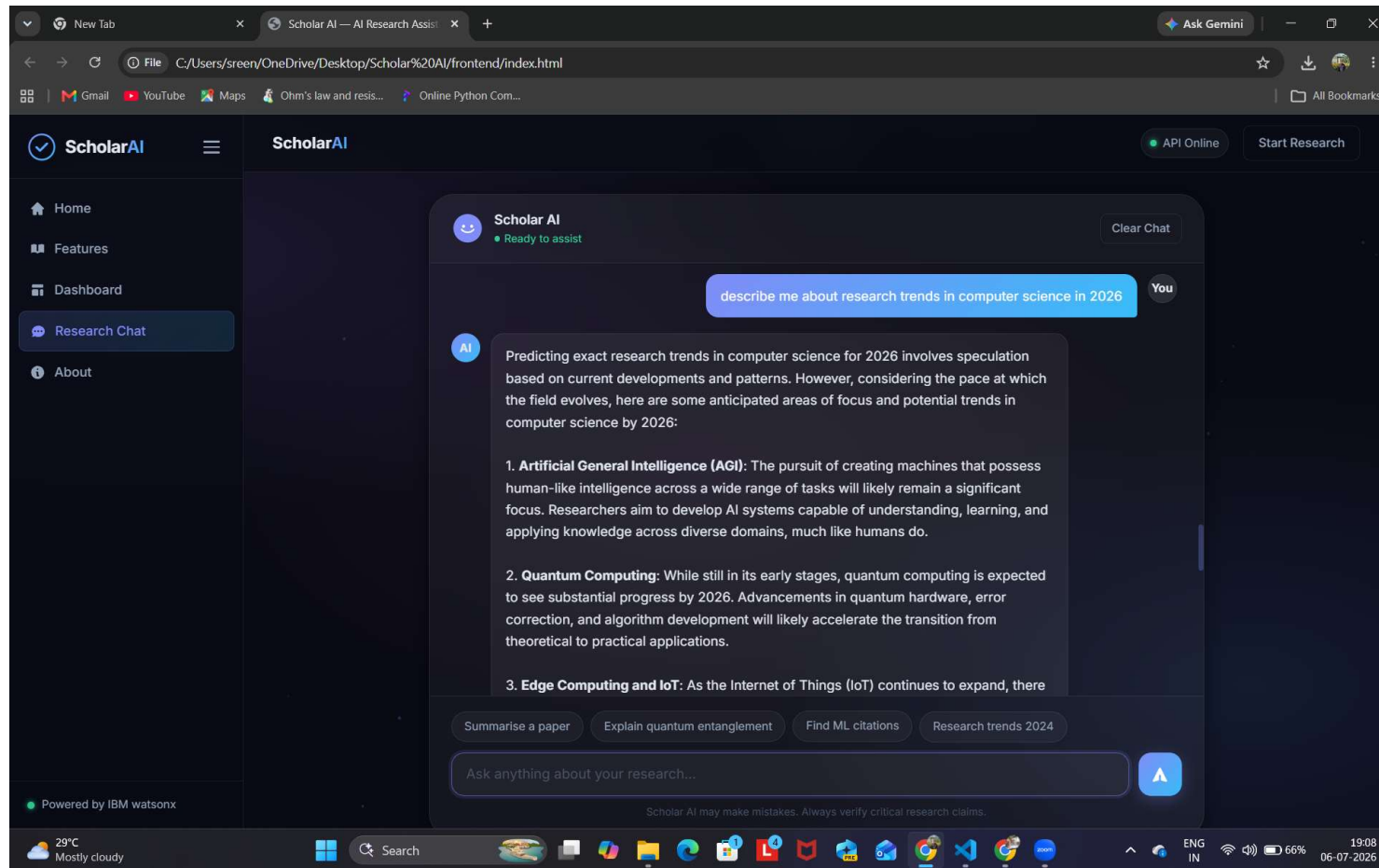
Completion date: 08 Jul 2026 (GMT)

Learning hours: 50 mins

Screenshots :-







Scholar AI — AI Research Assistant

Ask Gemini

C:/Users/sreen/OneDrive/Desktop/Scholar%20AI/frontend/index.html

GmailYouTubeMapsOhm's law and resis...Online Python Com...All Bookmarks

ScholarAI

HomeFeaturesDashboardResearch ChatAbout

API OnlineStart Research

Scholar AI

Ready to assist

Clear Chat

Research trends 2024You

AI

Predicting future research trends involves speculation based on current developments and patterns. As we approach 2024, some emerging trends in various fields of science and technology may include:

1. Artificial Intelligence and Machine Learning:
- Progress in explainable AI to address ethical concerns and improve transparency.
- Increased focus on lifelong and continual learning algorithms.
- Development of AI systems capable of commonsense reasoning.
- Growth in application of AI in healthcare, particularly for disease diagnosis and personalized treatment plans.

2. Quantum Computing:
- Advancements in error correction methods for quantum computers.
- Development of practical quantum algorithms for real-world problems.
- Integration of quantum computing with classical computing for hybrid systems.

Summarise a paper

Explain quantum entanglement

Find ML citations

Research trends 2024

Ask anything about your research...

Ask Gemini

Powered by IBM watsonx

Scholar AI may make mistakes. Always verify critical research claims.

29°C Mostly cloudy

Search

ENG IN

67%

19:07 06-07-2026

File Edit Selection View Go Run Terminal Help

Scholar AI

EXPLORER

SCHOLAR AI

assets

backend

__pycache__

config.cpython-313.pyc

services

__pycache__

__init__.py

orchestrate.py

venv

.env

.env.example

app.py

config.py

requirements.txt

frontend

index.html

script.js

style.css

screenshots

.gitignore

README.md

index.html

README.md

app.py

frontend > index.html > html > body > script

```
2 <html lang="en">
13 <body>
82   <main class="main-content" id="main-content">
296     <section class="about-section" id="about" aria-labelledby="about-heading">
297       <div class="about-grid">
321         <div class="stack-card">
322           <span class="stack-icon"><img alt="Vanilla JS icon" data-bbox="322 297 341 316"/></span>
323           <span class="stack-name">Vanilla JS</span>
324           <span class="stack-desc">Dynamic frontend</span>
325         </div>
326         <div class="stack-card">
327           <span class="stack-icon"><img alt="Enterprise Security icon" data-bbox="327 368 346 387"/></span>
328           <span class="stack-name">Enterprise Security</span>
329           <span class="stack-desc">IBM-grade</span>
330         </div>
331       </div>
332     </div>
333   </section>
334
335   <!-- FOOTER -->
336   <footer class="site-footer" role="contentinfo">
337     <div class="footer-inner">
338       <div class="footer-brand">
339         Scholar&ltspan class="logo-accent">AI</span>
340         <span class="footer-tagline">IBM Internship Project · 2026</span>
341       </div>
342       <p class="footer-copy">Built with ❤️ using IBM watsonx Orchestrate. For educational purposes.</p>
343     </div>
344   </footer>
345 </body>
346 </html>
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

(venv) PS C:\Users\sreen\OneDrive\Desktop\Scholar AI\backend> python app.py

2026-07-06 19:12:46 [INFO] __main__ - GET /health 200 (2ms)

2026-07-06 19:12:46 [INFO] werkzeug - 127.0.0.1 - [06/Jul/2026 19:12:46] "GET /health HTTP/1.1" 200 -

2026-07-06 19:12:55 [INFO] __main__ - GET /health

2026-07-06 19:12:55 [INFO] __main__ - GET /health 200 (0ms)

2026-07-06 19:12:55 [INFO] werkzeug - 127.0.0.1 - [06/Jul/2026 19:12:55] "GET /health HTTP/1.1" 200 -

2026-07-06 19:13:06 [INFO] __main__ - GET /health

2026-07-06 19:13:06 [INFO] __main__ - GET /health 200 (0ms)

2026-07-06 19:13:06 [INFO] werkzeug - 127.0.0.1 - [06/Jul/2026 19:13:06] "GET /health HTTP/1.1" 200 -

2026-07-06 19:13:15 [INFO] __main__ - GET /health

2026-07-06 19:13:15 [INFO] __main__ - GET /health 200 (0ms)

2026-07-06 19:13:15 [INFO] werkzeug - 127.0.0.1 - [06/Jul/2026 19:13:15] "GET /health HTTP/1.1" 200 -

OUTLINE

TIMELINE

Ln 348, Col 36

Spaces: 2

UTF-8

LF

HTML

File Edit Selection View Go Run Terminal Help

Scholar AI

EXPLORE

SCHOLAR AI

assets

backend

__pycache__

config.cpython-313.pyc

services

__pycache__

init.py

orchestrate.py

venv

.env

.env.example

app.py

config.py

requirements.txt

frontend

index.html

script.js

style.css

screenshots

.gitignore

README.md

OUTLINE

TIMELINE

backend > app.py

32 def create_app(config: Config | None = None) -> Flask:

55 orchestrate = OrchestrateService(cfg)

56

57 # --- Request lifecycle hooks ---

58 @app.before_request

59 def log_request():

60 request.start_time = time.time()

61 logger.info("%s %s", request.method, request.path)

62

63 @app.after_request

64 def log_response(response):

65 duration_ms = round((time.time() - request.start_time) * 1000)

66 logger.info("%s %s %d (%dms)",

67 request.method, request.path,

68 response.status_code, duration_ms)

69 return response

70

71 #

72 # ROUTES

73 #

74

75 # --- GET / ---

76 @app.route("/", methods=["GET"])

77 def index():

78 """Service information endpoint."""

79 return jsonify({

80 "service": "Scholar AI",

81 "version": "1.0.0",

82 "description": "IBM watsonx-powered AI Research Assistant",

83 "powered_by": "IBM watsonx Orchestrate",

84 "docs": "/health",

85 "timestamp": datetime.utcnow().isoformat() + "Z",

86 }, 200

87

88 # --- GET /health ---

89 @app.route("/health", methods=["GET"])

90 def health():

91 """

92 Health check endpoint.

93 Returns service status and IBM watsonx connectivity information.

94 """

95 orchestrate_status = orchestrate.ping()

CHAT

SESSIONS

Ln 14, Col 12 Spaces: 4 UTF-8 LF Python

29°C Mostly cloudy

18:02 06-07-2026

File Edit Selection View Go Run Terminal Help

Scholar AI

08 06-07-2026

EXPLORER

SCHOLAR AI

assets

backend

__pycache__

config.cpython-313.pyc

services

__pycache__

__init__.py

orchestrate.py

venv

.env

.env.example

app.py

config.py

requirements.txt

frontend

index.html

script.js

style.css

screenshots

.gitignore

README.md

OUTLINE

TIMELINE

backend > app.py

```
1 """
2 Scholar AI - app.py
3 Flask REST API backend for Scholar AI research assistant.
4 IBM watsonx Orchestrate integration-ready.
5
6 Endpoints:
7 GET /          -> Service info
8 GET /health    -> Health check
9 POST /chat     -> Chat with Scholar AI (IBM watsonx)
10
11 """
12
13 import logging
14 import time
15 from datetime import datetime
16
17 from flask import Flask, jsonify, request
18 from flask_cors import CORS
19
20 from config import Config
21 from services.orchestrate import OrchestrateService
22
23 # --- Logging configuration ---
24 logging.basicConfig(
25     level=logging.INFO,
26     format="%(asctime)s [%(levelname)s] %(name)s - %(message)s",
27     datefmt="%Y-%m-%d %H:%M:%S",
28 )
29 logger = logging.getLogger(__name__)
30
31 # --- Application factory ---
32 def create_app(config: Config | None = None) -> Flask:
33     """
34     Create and configure the Flask application.
35
36     Args:
37         config: Optional Config instance. Defaults to Config() which
38                 reads from environment variables / .env file.
39
40     Returns:
41         Configured Flask app instance.
42     """
```

CHAT

SESSIONS

Tip: Try the [Plan agent](#) to research and plan before implementing changes.

app.py

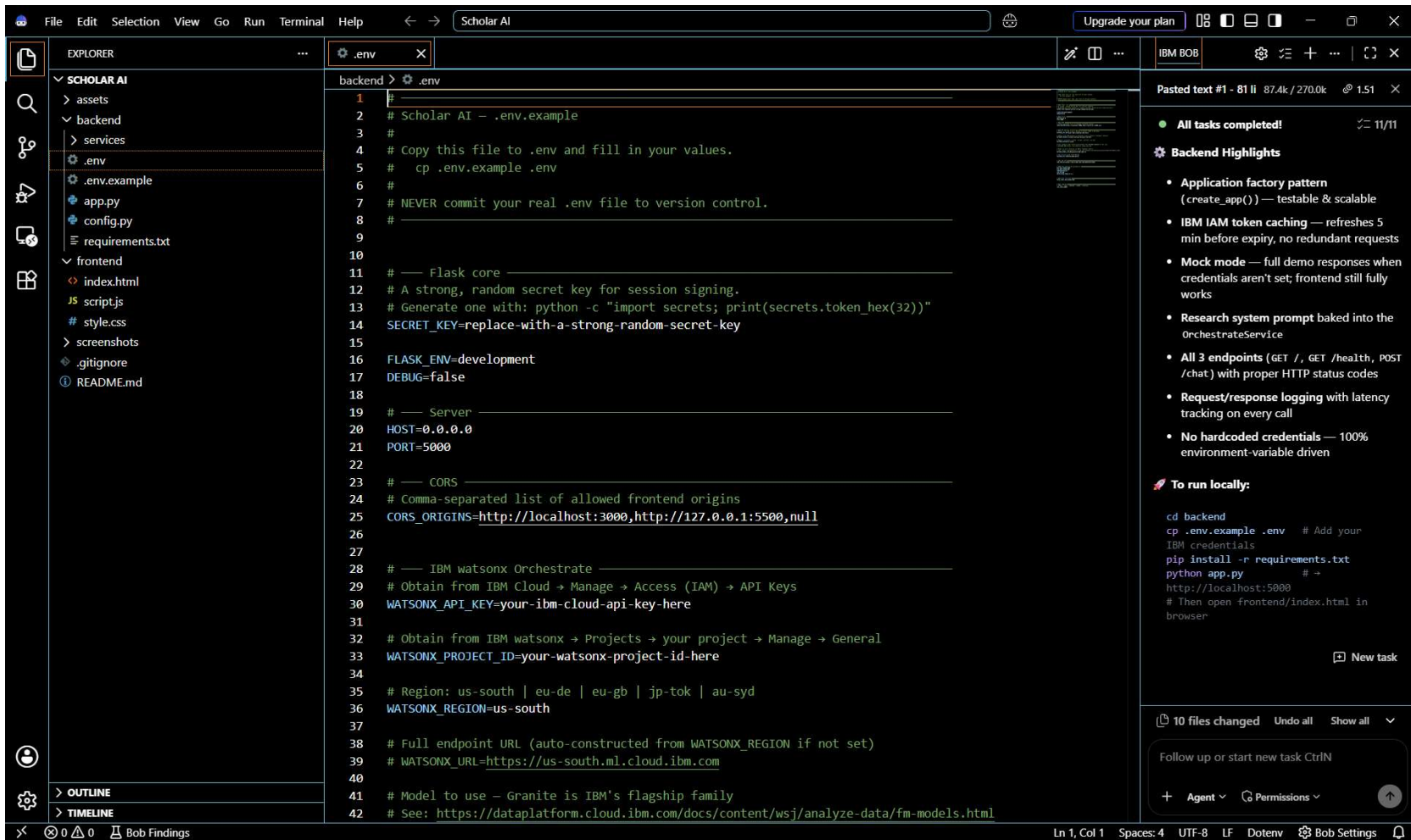
Describe what to build

+ | | Auto | ↵

Local | Default Approvals

Ln 14, Col 12 Spaces: 4 UTF-8 LF {} Python

29°C Mostly cloudy Search 89% 18:02 06-07-2026



File Edit Selection View Go Run Terminal Help

Scholar AI

Upgrade your plan

EXPLORER

SCHOLAR AI

assets

backend

services

.env.example

app.py

config.py

requirements.txt

frontend

index.html

script.js

style.css

screenshots

.gitignore

README.md

OUTLINE

TIMELINE



Show All Commands

Ctrl + Shift + P

Start Debugging

F5

Go to File

Ctrl + P

IBM BOB

Pasted text #1 - 81 li 87.4k / 270.0k 1.51 X

All tasks completed! 11/11

Backend Highlights

- Application factory pattern (create_app()) — testable & scalable
- IBM IAM token caching — refreshes 5 min before expiry, no redundant requests
- Mock mode — full demo responses when credentials aren't set; frontend still fully works
- Research system prompt baked into the OrchestrateService
- All 3 endpoints (GET /, GET /health, POST /chat) with proper HTTP status codes
- Request/response logging with latency tracking on every call
- No hardcoded credentials — 100% environment-variable driven

To run locally:

```
cd backend
cp .env.example .env # Add your IBM credentials
pip install -r requirements.txt
python app.py # → http://localhost:5000
# Then open frontend/index.html in browser
```

New task

10 files changed Undo all Show all

Follow up or start new task Ctrl+N

Agent

Permissions

Bob Findings

Bob Settings

Light rain Tomorrow

Search

ENG IN

77%

20:10 05-07-2026

Thank You!

Thank you for your time and interest.